

Author

- Laurent d'Orazio, Univ Rennes, CNRS, IRISA

1 Introduction

Company
id:int
guid:string
name:string
abbreviation:string
creation:string
city:string
state:string
country:string
image:string

Figura 1: Data schema Company

Game
id:int
guid:string
title:string
release:string
cover:string

Figura 2: Data schema Game

Platform
id:int
guid:string
name:string
abbreviation:string
release:string
price:string
company:int
image:string

Figura 3: Data schema Platform

The context to be considered is games. Figure 1, 2 and 3 present the data schemas. In particular they focus on a set companies, games and platforms.

2 Deployment

Image

```
docker pull apache/hadoop:3.3.5
```

```
cd <working_dir>
```

Edit files
config

```
CORE-SITE.XML_fs.default.name=hdfs://namenode
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=1
MAPRED-SITE.XML_mapreduce.framework.name=yarn
MAPRED-SITE.XML_yarn.app.mapreduce.am.env=HADOOP_MAPRED_HOME=$HADOOP_HOME
MAPRED-SITE.XML_mapreduce.map.env=HADOOP_MAPRED_HOME=$HADOOP_HOME
MAPRED-SITE.XML_mapreduce.reduce.env=HADOOP_MAPRED_HOME=$HADOOP_HOME
YARN-SITE.XML_yarn.resourcemanager.hostname=resourcemanager
YARN-SITE.XML_yarn.nodemanager.pmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.delete.debug-delay-sec=600
YARN-SITE.XML_yarn.nodemanager.vmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.aux-services=mapreduce_shuffle
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.maximum-applications=10000
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.maximum-am-resource-percent=0.1
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.resource-calculator=org.apache.hadoop.yarn.util.resource.DefaultResourceCalculator
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.queues=default
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.capacity=100
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.user-limit-factor=1
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.maximum-capacity=100
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.state=RUNNING
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.acl_submit_applications=*
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.acl_administer_queue=*
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.node-locality-delay=40
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.queue-mappings=
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.queue-mappings-override.enable=false
```

`docker-compose.yml`

```
version: "2"
services:
  namenode:
    image: apache/hadoop:3.3.5
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - 9870:9870
    env_file:
      - ./config
    environment:
      ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"
  datanode:
    image: apache/hadoop:3.3.5
    command: ["hdfs", "datanode"]
    env_file:
      - ./config
  resourcemanager:
    image: apache/hadoop:3.3.5
    hostname: resourcemanager
    command: ["yarn", "resourcemanager"]
    ports:
      - 8088:8088
    env_file:
      - ./config
    volumes:
      - ./test.sh:/opt/test.sh
  nodemanager:
    image: apache/hadoop:3.3.5
    command: ["yarn", "nodemanager"]
    env_file:
      - ./config
```

Docker compose

```
docker-compose up -d
```

```
docker exec -it hadoop-namenode-1 /bin/bash
```

3 Configuration

Create a Maven project (GroupId=what you want, ArtifactId example=MRGame)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com</groupId>
  <artifactId>hadoop_api_01082022</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <repositories>
    <repository>
      <id>apache</id>
      <url>https://maven.apache.org</url>
    </repository>
  </repositories>
  <dependencies>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-common</artifactId>
      <version>3.2.1</version>
    </dependency>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-hdfs</artifactId>
      <version>3.2.1</version>
    </dependency>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-hdfs-client</artifactId>
      <version>3.2.1</version>
    </dependency>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-mapreduce-client-core</artifactId>
      <version>3.2.1</version>
    </dependency>
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-mapreduce-client-shuffle</artifactId>
      <version>3.2.1</version>
    </dependency>
  </dependencies>
</project>
```

4 Tutorial

The following example enables to retrieve the number of companies par country.

4.1 Company

```
public class Studio {

    static String[] fields;
```

```

    public static void fromLine(String line){
        fields = line.split(";");
    }

    public static int getId() throws NumberFormatException{
        return Integer.parseInt(fields[0]);
    }

    public static String getCountry(){
        return fields[2];
    }
}

```

4.2 Map

```

import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class MoviesOMapper
extends Mapper<LongWritable, Text, Text, DoubleWritable>{

    private Text outKey = new Text();
    private DoubleWritable outValue = new DoubleWritable();

    public void map(LongWritable inKey, Text inValue, Context context)
    throws IOException, InterruptedException{
        try{
            if (inKey.get() == 0L) return;
            String line = inValue.toString();
            Studio.fromLine(line);
            String country=Studio.getCountry();
            outKey.set(country);
            outValue.set(new Double(1.0));
            context.write(outKey, outValue);
        }
        catch(Exception e){}
    }
}

```

4.3 Reduce

```

import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class MoviesOReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable>
{
    private Text outKey;
    private DoubleWritable outValue = new DoubleWritable();

    @Override
    public void reduce(Text inKey, Iterable<DoubleWritable> inValues, Context context)
        throws IOException, InterruptedException
    {

```

```

        outKey = inKey;
        int sum=0;
        for (DoubleWritable inValue : inValues) {
            sum++;
        }
        Double result = new Double(sum);
        outValue.set(result);
        System.out.println(outKey+" - "+sum);
        context.write(outKey, outValue);
    }
}

```

4.4 Driver

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class Movies0Driver
    extends Configured
    implements Tool
{
    @Override
    public int run(String[] args) throws Exception
    {
        // time to start
        Long initTime = System.currentTimeMillis();

        //parameters checking
        if (args.length != 2) {
            System.err.println("Usage: ModeleMapReduceDriver <inputpath> <outputpath>");
            System.exit(-1);
        }

        //map-reduce job creation
        //Configuration conf = this.getConf();
        //local version
        Configuration conf = new Configuration();
        conf.addResource(new
            Path("/Users/ldorazio/Documents/travail/developpement/hadoop-3.2.1/etc/hadoop/core-site.xml"));

        Job job = Job.getInstance(conf, "ModeleMapReduce Job");
        job.setJarByClass(Movies0Driver.class);

        //Mapper and Reducer classes definition
        job.setMapperClass(Movies0Mapper.class);
        /*job.setCombinerClass(ModeleMapReduceCombiner.class);*/
        job.setReducerClass(Movies0Reducer.class);

        ///input definition
        //TextInputFormat => keys=LongWritable, values=Text
        FileInputFormat.setInputDirRecursive(job, false); // true if folder
        FileInputFormat.addInputPath(job, new Path(args[0]));
        job.setInputFormatClass(TextInputFormat.class);
    }
}

```

```

//mapper output=reducer input
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(DoubleWritable.class);

//output definition: folder and key value types
FileOutputFormat.setOutputPath(job, new Path(args[1]));
job.setOutputFormatClass(TextOutputFormat.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(DoubleWritable.class);

//run the job and measure of the response time
Long startTime = System.currentTimeMillis();
boolean success = job.waitForCompletion(true);
Long endTime = System.currentTimeMillis();
System.out.println("Job Duration seconds: " + ((endTime-startTime)/1000L));
System.out.println("Total Duration seconds: " + ((endTime-initTime)/1000L));
return success ? 0 : 1;
}

public static void main(String[] args) throws Exception
{
    args=new String[2];
    args[0]="../data/movies/studios.csv";
    args[1]="results";
    //prepare and run next job
    MoviesODriver driver = new MoviesODriver();
    int exitCode = ToolRunner.run(driver, args);
    System.exit(exitCode);
}
}

```

5 Activities

1. Aggregation: Number of movies per year.*
2. Aggregation: Number of movies per year.
3. Project: Create a new project to list the names of the studios.*
4. Selection: studios where country is FR.
5. Aggregation and comparaison: Longest movie per year.
6. Comparaison: Last movie.
7. Comparison: country with the maximum number of studios.

6 Monitoring

<http://localhost:9870>

7 References

- **Lecture:** https://perso.univ-rennes1.fr/laurent.dorazio/data/teachings/big_data/big_data_mapreduce.pdf
- **Apache Hadoop:** <https://hadoop.apache.org/>
- **Pierre Nerzic class on Big Data:** <https://perso.univ-rennes1.fr/pierre.nerzic/Hadoop/>